

ECE 350
Real-time
Operating
Systems



Lecture 1: Introduction

Prof. Seyed Majid Zahedi

<https://ece.uwaterloo.ca/~smzahedi>

Outline

- How do things work in ECE 350?
- What is a real-time system?
- What is an operating system?
- What makes operating systems so exciting?

Useful Links

- Course webpage

<https://ece.uwaterloo.ca/~smzahedi/crs/ece350>

- Course on Piazza

<https://piazza.com/uwaterloo.ca/summer2026/ece350>

- Anonymous feedback form

<https://forms.gle/7BRouEH92BtTMbfb9>

Useful Links

- Course webpage



- Course on Piazza



- Anonymous feedback form



ECE 350 is a Class About...

- Design of key systems abstractions that have emerged over time
 - Processes, threads, events, address spaces, file systems, sockets, transactions, key-value stores, etc.
- Tradeoffs surrounding these designs
- Their efficient implementation
 - Including hardware support that makes them possible and practical
- And how to use them effectively

Why Take ECE 350?

Why Learn About OS?

- Some of you will design and build parts of (real-time) operating systems
- Many of you will create systems that use OS concepts
 - Whether you build hardware or software
 - Concepts and design patterns appear at many levels
- All of you will write programs that use OS abstractions
 - The better you understand them, the better you use them

PAY ATTENTION!

**THIS IS THE IMPORTANT STUFF.
SERIOUSLY GUYS.**

memegenerator.net

Evaluation

- Final: 50%
- Lab projects: 30%
- Midterm: 20%

Labs



Start Early!



- Time/work estimation is hard
 - Programmers are eternal optimists (it will only take two days!)
 - This is why we bug you about starting the project early

Techniques for Partitioning Tasks

- Functional
 - Person A implements threads, Person B implements semaphores, Person C implements locks...
 - Problem: Lots of communication across APIs
 - If B changes the API, A may need to make changes
- Task
 - Person A designs, Person B writes code, Person C tests
 - May be difficult to find right balance, but can focus on each person's strengths (Theory vs systems hacker)
 - Since debugging is hard, Microsoft has two testers for each programmer

Collaboration Policy

- Explaining concepts to someone in another group
- Discussing algorithms/testing strategies with other groups
- Helping debug someone else's code (in another group)
- Searching online for generic algorithms (e.g., hash table)
- Sharing code or test cases with another group
- Open-sourcing code (e.g., on GitHub) even after this term
- Copying OR reading another group's code or test cases
- Copying OR reading online code or test cases from prior years
- **Zero tolerance policy for plagiarism**
 - We use [Moss](#) and follow [UW Policy 71](#) for any single incident



WORK HARD



RELAX HARDER

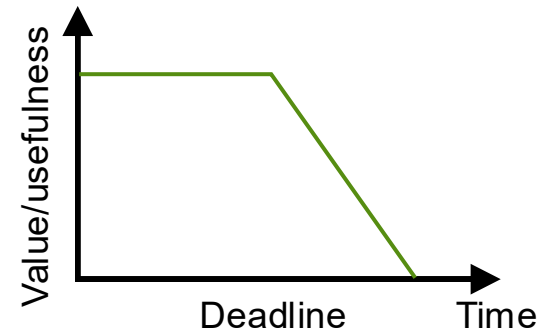
makeameme.org

What is a Real-time System?

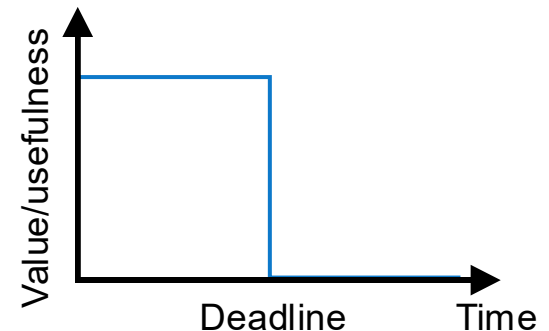
- Systems whose correctness depends on both their **temporal** aspects as well as their **functional** aspects
- Temporal aspects are typically specified through **deadline**
 - Results must be produced before deadline
 - Correct result produced too late is considered failure
- Real-time systems are typically embedded control systems
 - E.g., safety-critical systems
- OS must be carefully designed to support temporal properties

Types of Real-time Systems

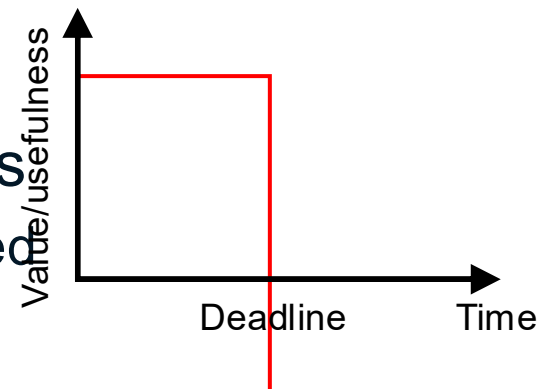
- **Soft**: must try to meet all deadlines
 - System does not fail if a few deadlines are missed



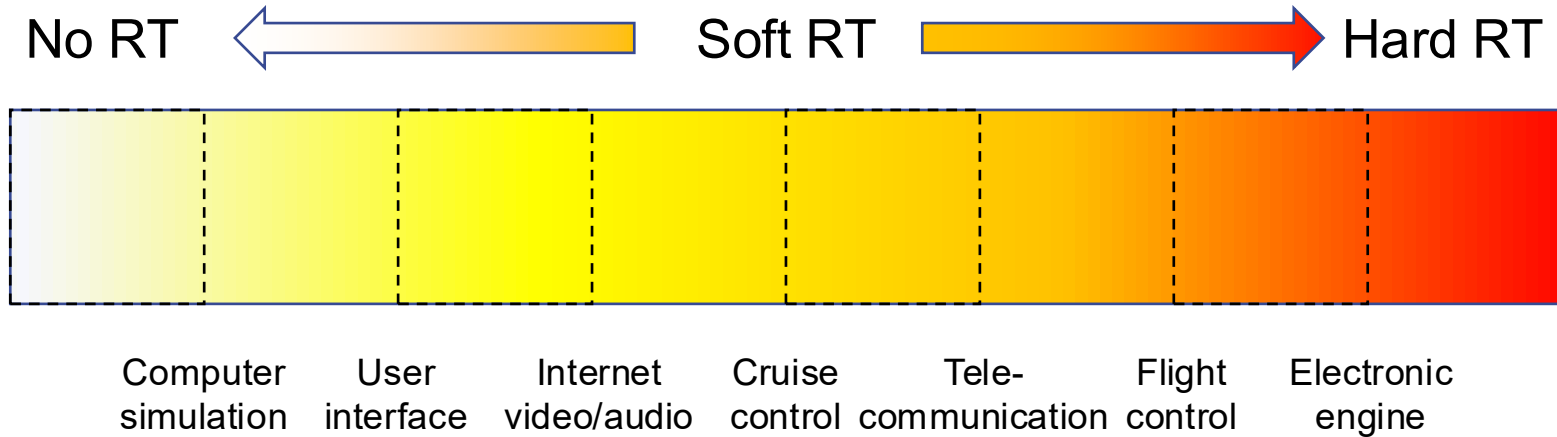
- **Firm**: result has no use outside deadline window
 - Tasks that fail are discarded



- **Hard**: must always meet all deadlines
 - System fails if deadline window is missed



Real-time Spectrum

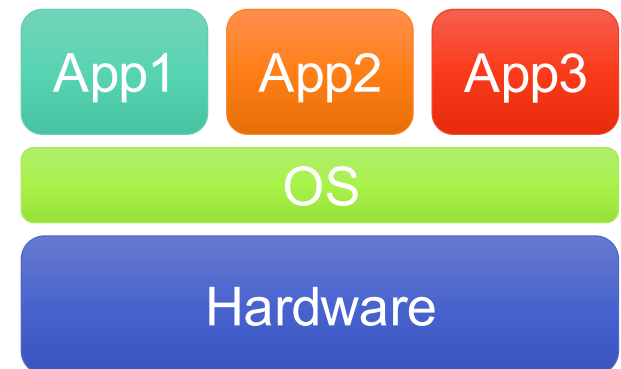


What is an Operating System?

- **No** universally accepted definition
- “Everything vendors ship when you order OS” is **good approximation**, but varies wildly
- “The one program running at all times on computer” is **kernel**
 - Everything else is either system program (ships with OS) or application program

What is an Operating System? (cont.)

- Special layer of software that provides applications access to hardware resources
 - **Abstract** view of complex hardware devices
 - **Protected** access to shared resources
 - Security and authentication
 - Communication amongst logical entities



What is an Operating System? (cont.)

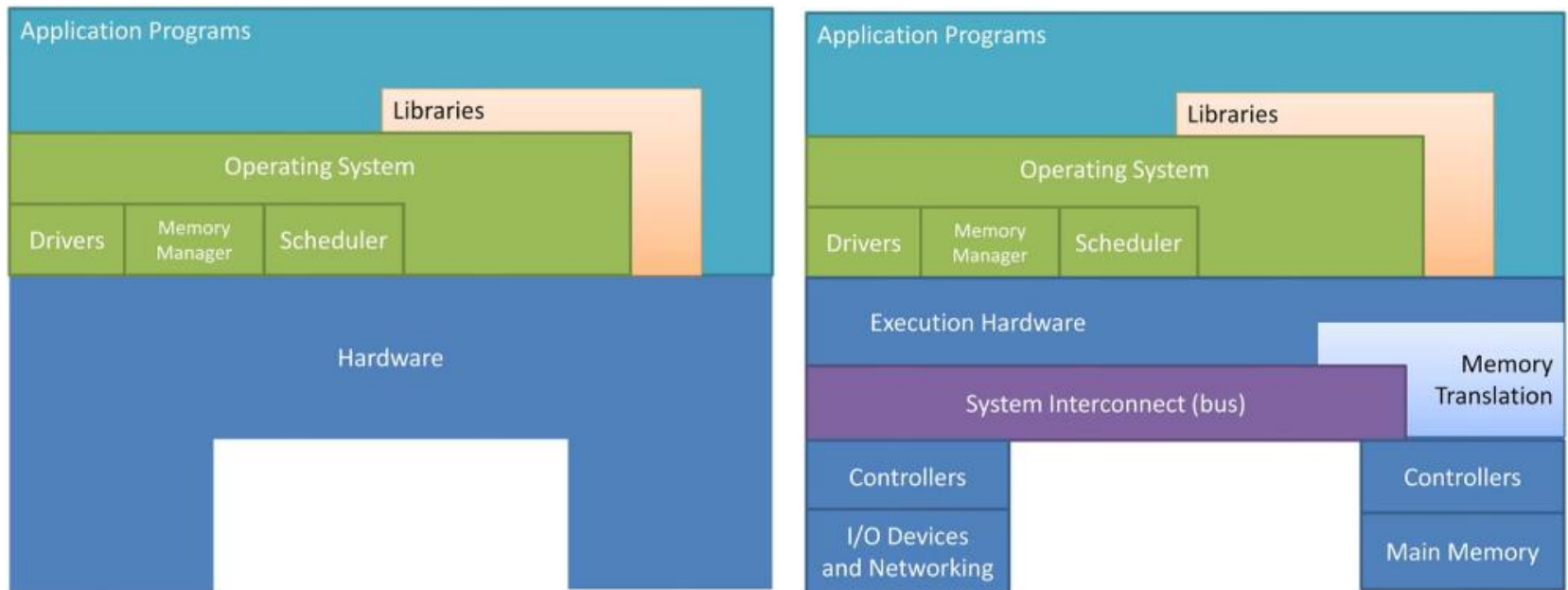
- Illusionist

- Provide clean, easy-to-use abstractions of physical resources

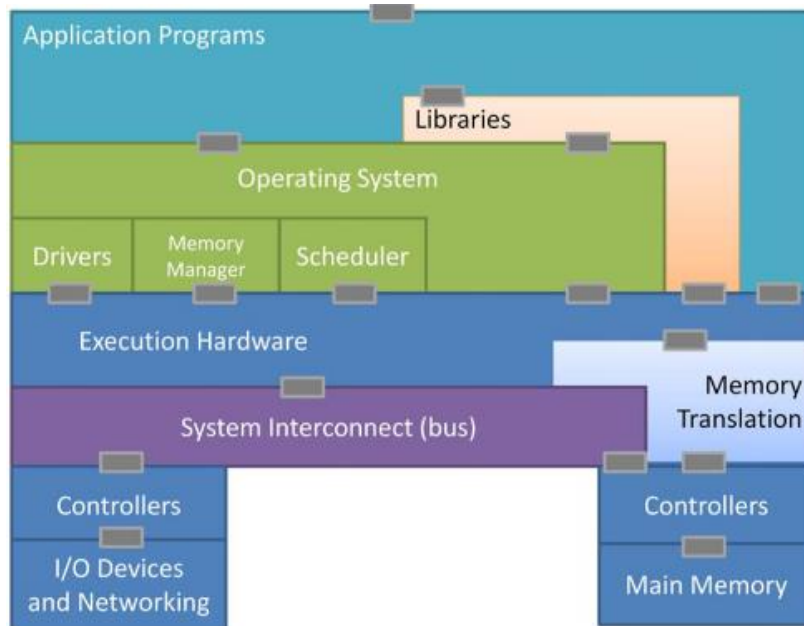


- Infinite memory, dedicated machine
 - Higher level objects: files, users, messages
 - Masking limitations, virtualization

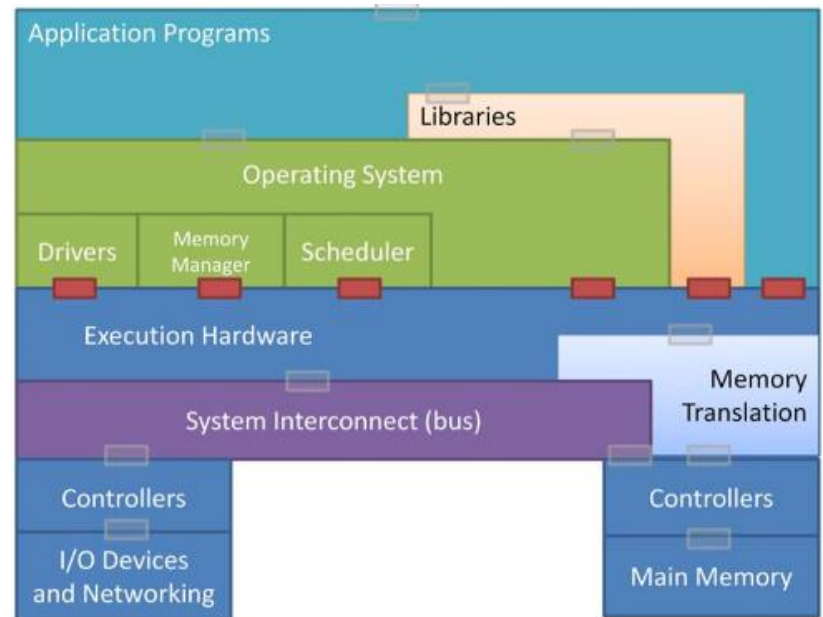
Hardware/Software Interface



Hardware/Software Interface (cont.)

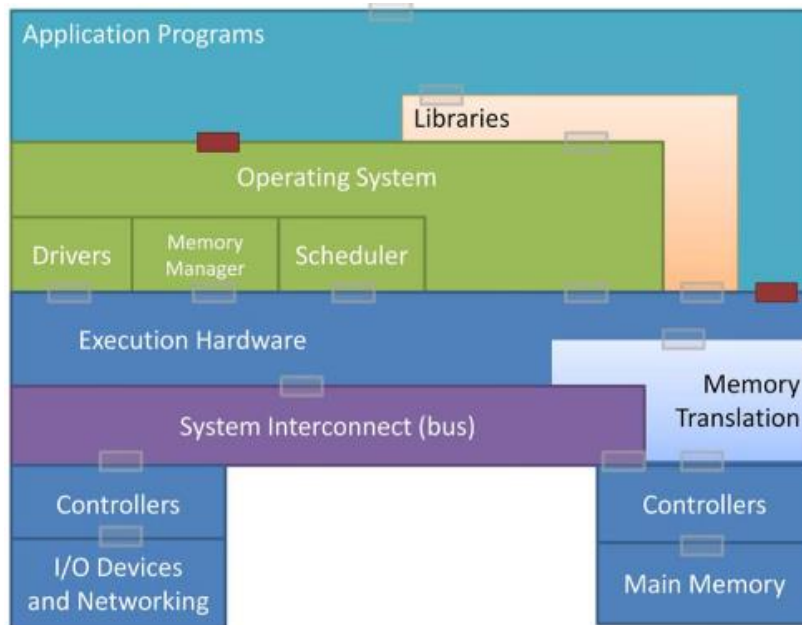


■ Interfaces

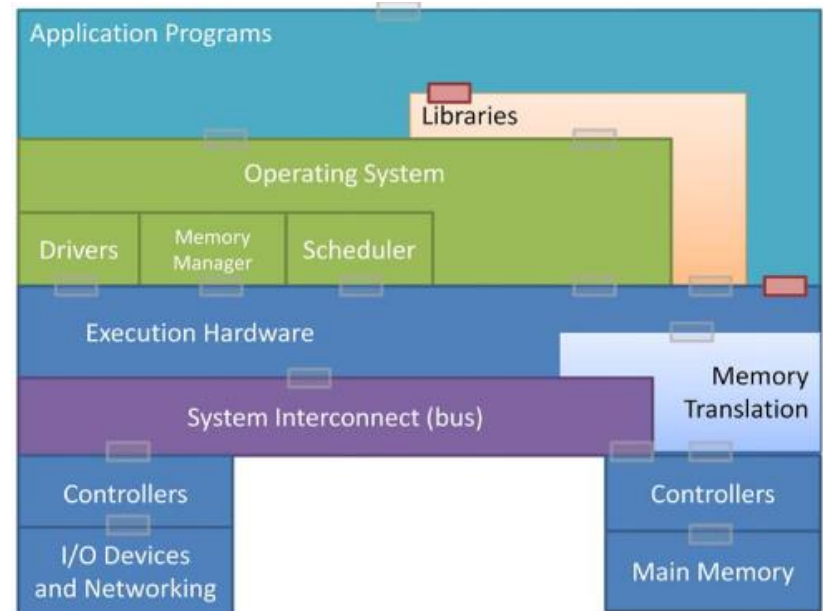


■ Instruction Set Architecture (ISA)

Hardware/Software Interface (cont.)

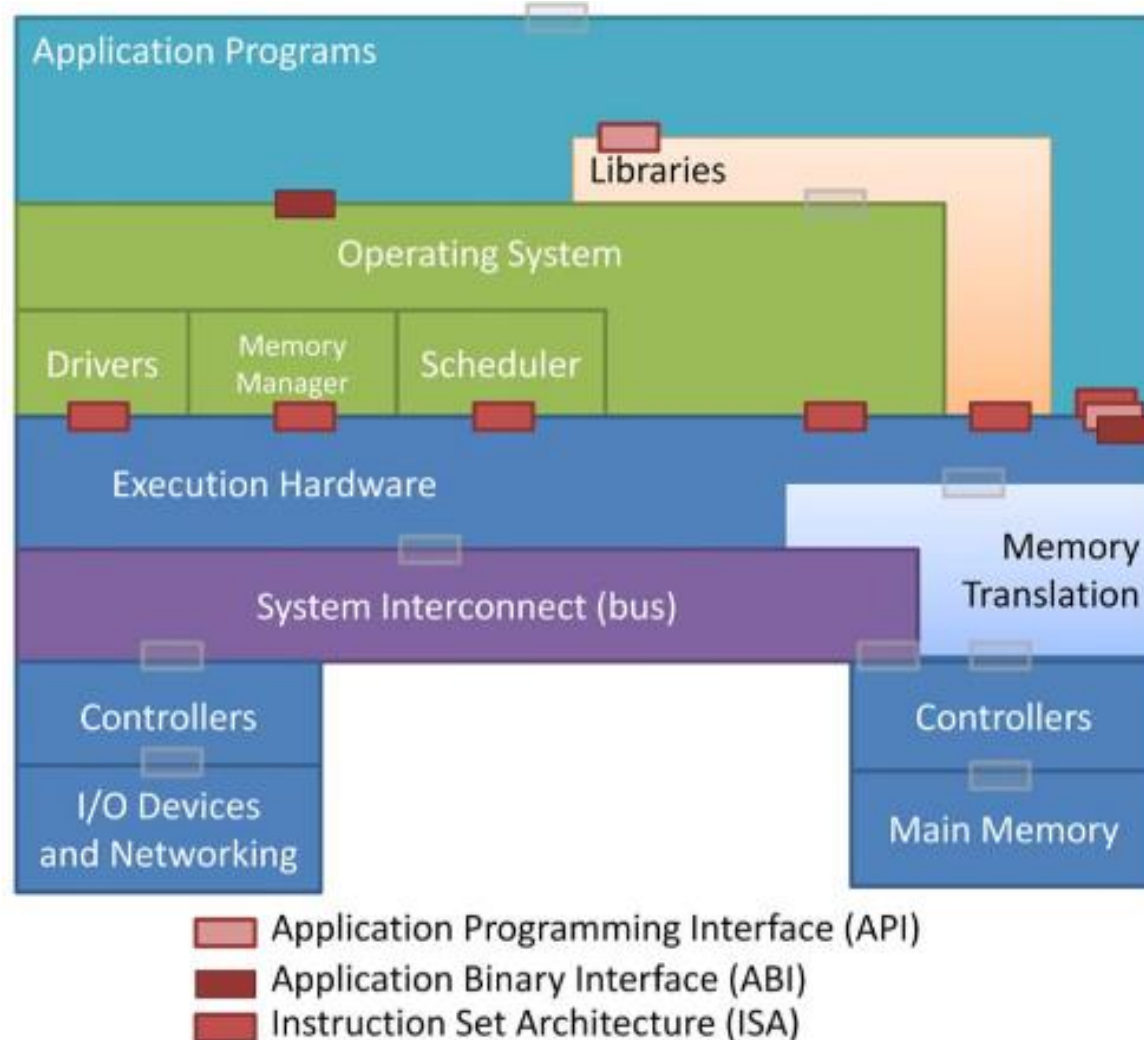


■ Application Binary Interface (ABI)



■ Application Programming Interface (API)

Hardware/Software Interface (cont.)



What is an Operating System? (cont.)

- Illusionist



- Provide clean, easy-to-use abstractions of physical resources
 - Infinite memory, dedicated machine
 - Higher level objects: files, users, messages
 - Masking limitations, virtualization

- Referee



- Provide protection, isolation, and sharing of resources
 - Resource allocation and communication

What is an Operating System? (cont.)

- Illusionist



- Provide clean, easy-to-use abstractions of physical resources
 - Infinite memory, dedicated machine
 - Higher level objects: files, users, messages
 - Masking limitations, virtualization

- Referee



- Provide protection, isolation, and sharing of resources
 - Resource allocation and communication

- Glue



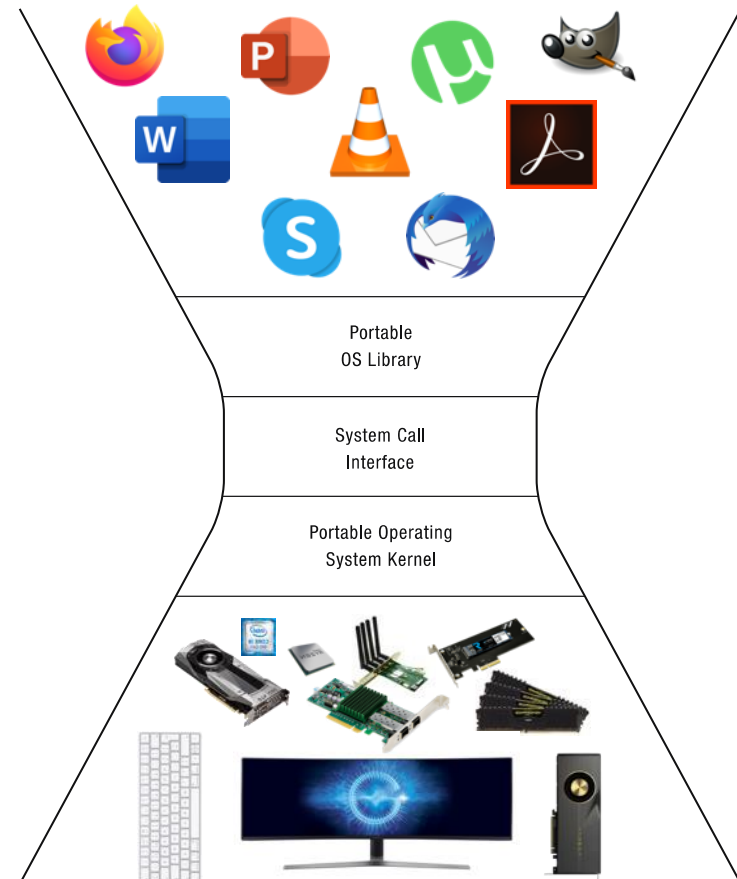
- Provide common services
 - Storage, window system, networking, sharing, authorization
 - Look and feel

OS Basics: Hardware Support

- OS bottom line is to support applications!
 - OS itself is **incidental**
 - Ideally, OS should have very low performance overhead over raw hardware
- OS relies on HW support to provide abstractions **efficiently**
 - Dual-mode operation, interrupts, traps, precise exceptions, memory management unit, translation lookaside buffer, etc.
- HW support and OS design continue to co-evolve...
 - ... as hardware performance improves (e.g., faster storage/network),
...
 - ... and application requirements change
 - What we study in this class is result of decades of co-evolution!

What do Operating Systems do?

- Provide abstractions to applications
 - File systems
 - Processes, threads
 - Virtual memory
 - Naming system, ...
- Manage diverse resources
 - Memory, CPU, storage, ...
- Achieves above by implementing specific algorithms and techniques
 - Scheduling
 - Concurrency
 - Transactions
 - Security



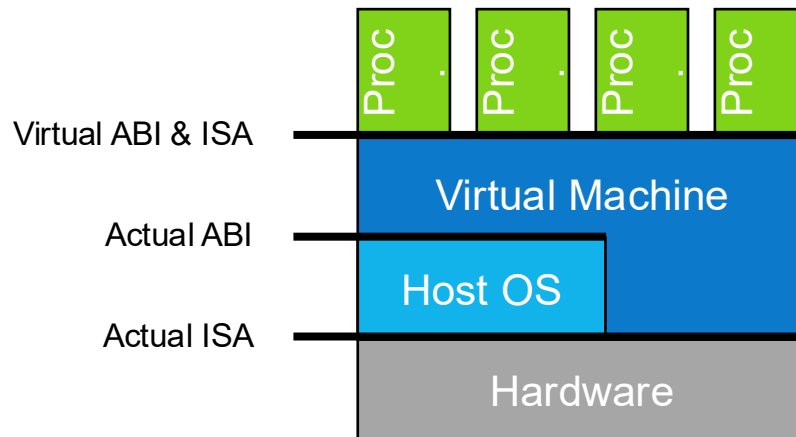
Virtual Machines (VMs)

- Software that emulates physical machine
 - Gives programs **illusion** that they run on physical machine
 - Provides platform that is independent of actual underlying hardware
 - Makes it look like hardware has features programs want
- Two types of virtual machines
 - Process VM: supports execution of single program (e.g., Java)
 - System VM: supports execution of entire OS (e.g., VMWare Fusion, Virtual box, Parallels Desktop, Xen)



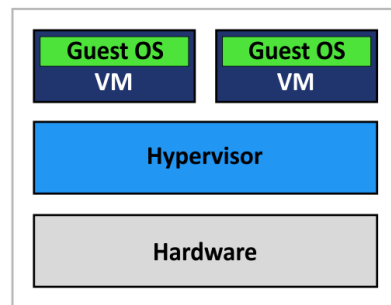
Process VMs

- Runs processes
 - Abstracts underlying OS and hardware
 - Provides platform-independent environment
 - E.g., Java virtual machine, .NET framework

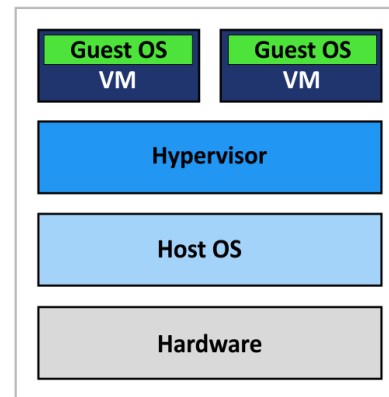


System Virtual Machines: Layers of OSes

- Runs OSes
 - Useful for OS development and testing programs on other OSes
- Hypervisors create and run virtual machines
- **Type-1** hypervisors allocate HW to VMs in addition to managing them
 - E.g., Xen, VMWare ESXi
- **Type-II** hypervisors rely on host OS for HW management
 - E.g., Virtual



Type 1 Hypervisor
(Bare-Metal Architecture)



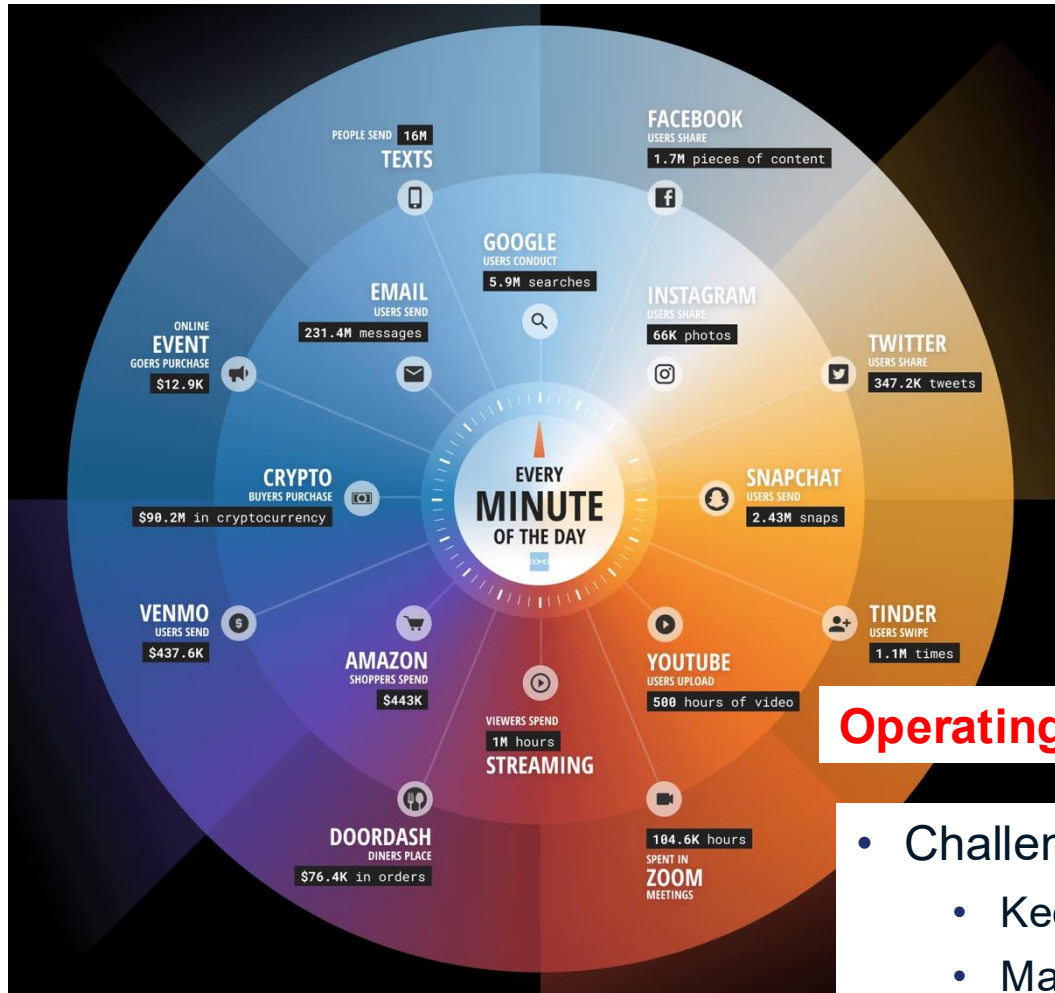
Type 2 Hypervisor
(Hosted Architecture)

Containers: Low-weight Alternatives to Full-system Virtualization

- Provide OS virtualization above single shared kernel
 - Do not provide *full-machine* virtualization
 - Each VM has illusion of running on isolated machine
 - Each container has illusion of running on isolated OS
- Use OS constructs to provide *sand boxes* for execution
 - E.g., Linux cgroups, namespaces, etc.
- Can run on bare metal OS, or atop of OS running in VM
- **OS containers:** multiple applications run in same container
 - E.g., LXC, OpenVZ, FreeBSD Jail
- **Application containers:** each application has its own container
 - E.g., Docker, rkt



What Makes Operating Systems so Exciting and Challenging?

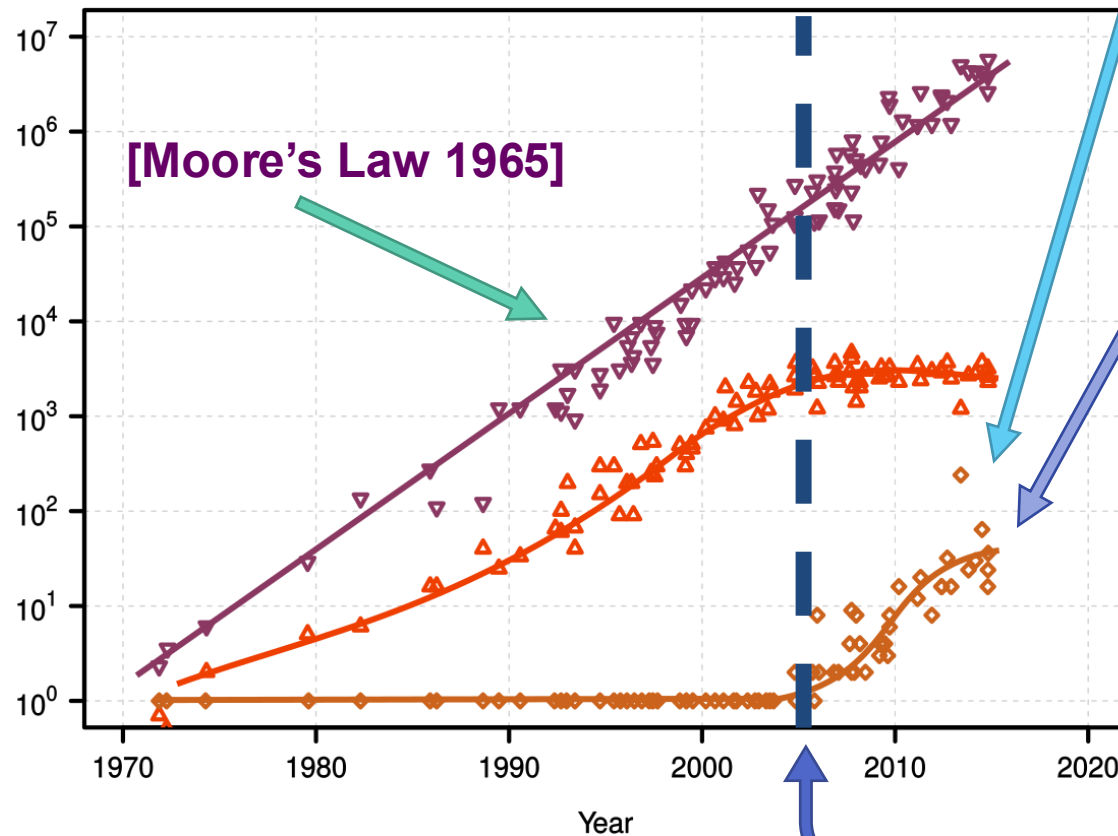


Operating systems are at the heart of it all

- Challenges
 - Keeping up with evolving HW
 - Managing ever-growing complexity of SW

Technology Trends

How do we program these?
Parallelism must be exploited at all levels



Transistors

Dark Silicon [Esmaeilzadeh et al. 2011]
Multiprocessor melts if all cores simultaneously run at full capacity

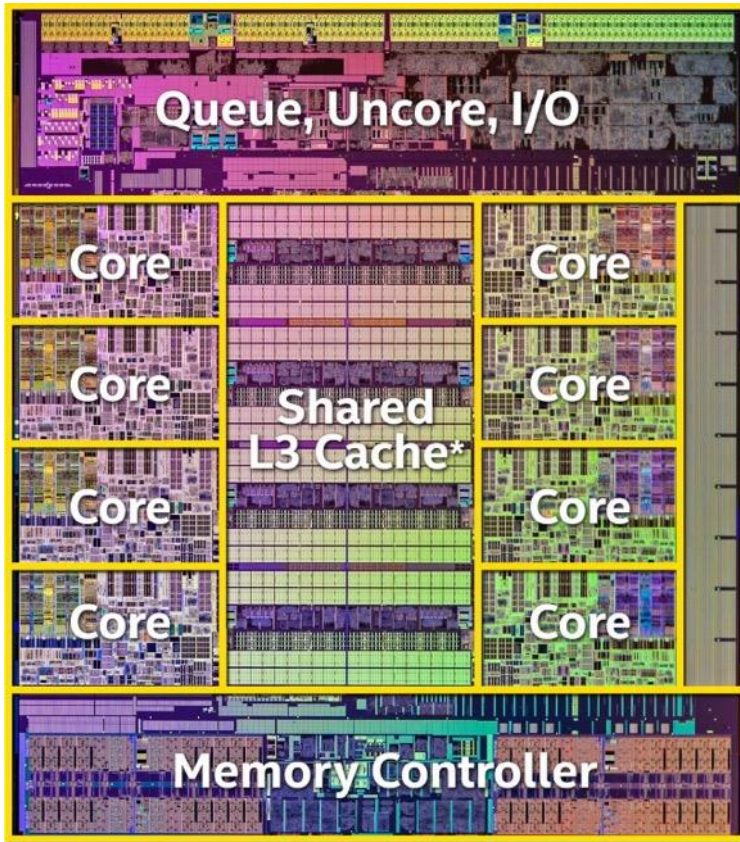
Frequency (MHz)

Number of Cores

End of Dennard Scaling
[R. Dennard et al. 1974]

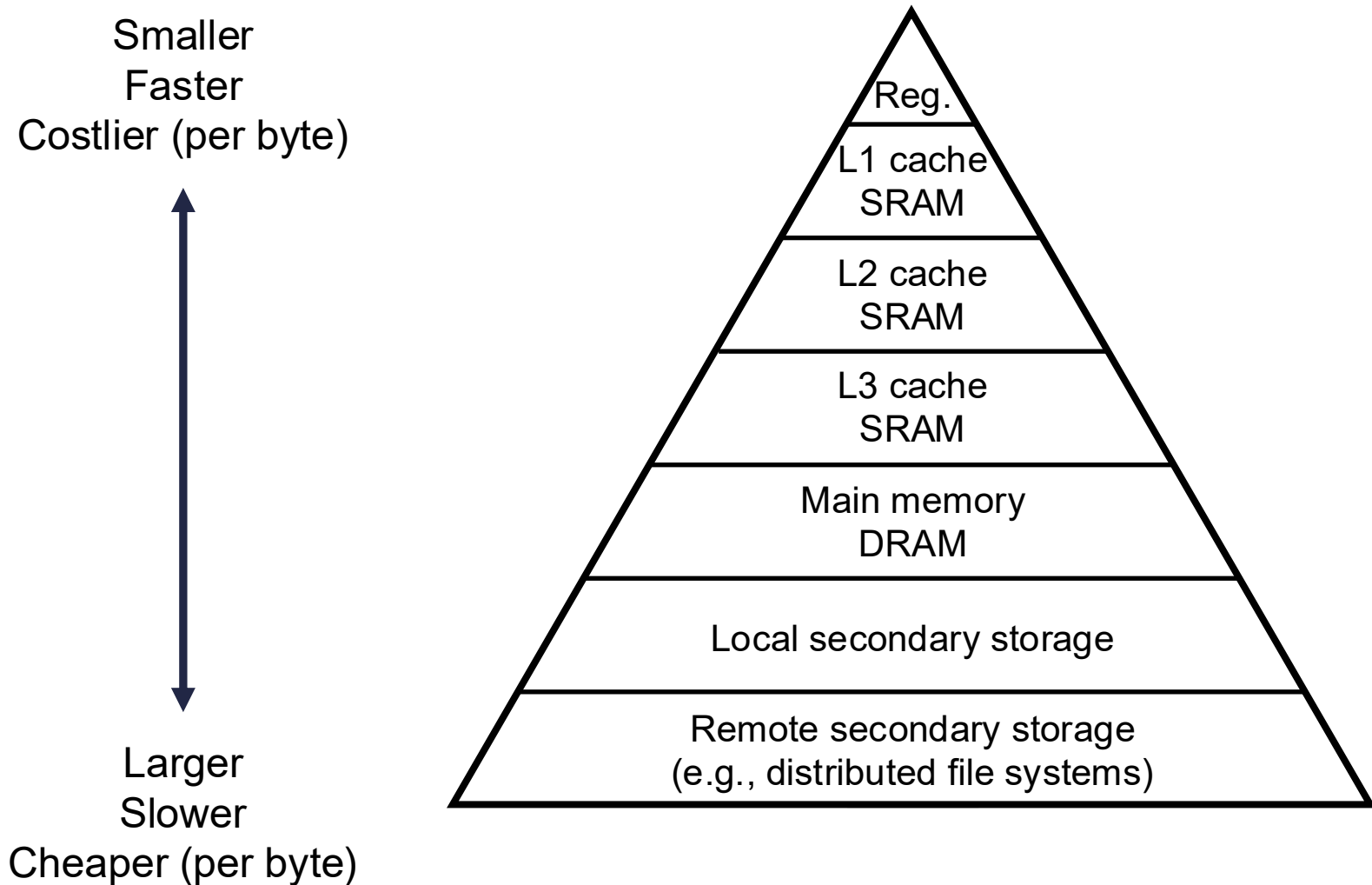
Modern Processors

Intel Haswell E



- Intel Xeon Platinum 9282
 - 14nm processor
 - 56 cores, 112 threads
 - 1.75MB data and ins. L1 cache
 - 56MB L2 cache
 - 77MB shared L3 cache
 - 8B transistors
- AMD EPYC 7H12
 - 7nm processor
 - 64 cores, 128 threads
 - 2MB data and ins. L1 cache
 - 32MB L2 cache
 - 256MB shared L3 cache
 - 4.8B transistors

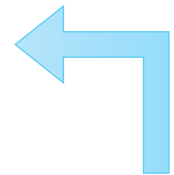
Memory Hierarchy



Numbers Everyone Should Know [Jeff

Dean, 2009]

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns



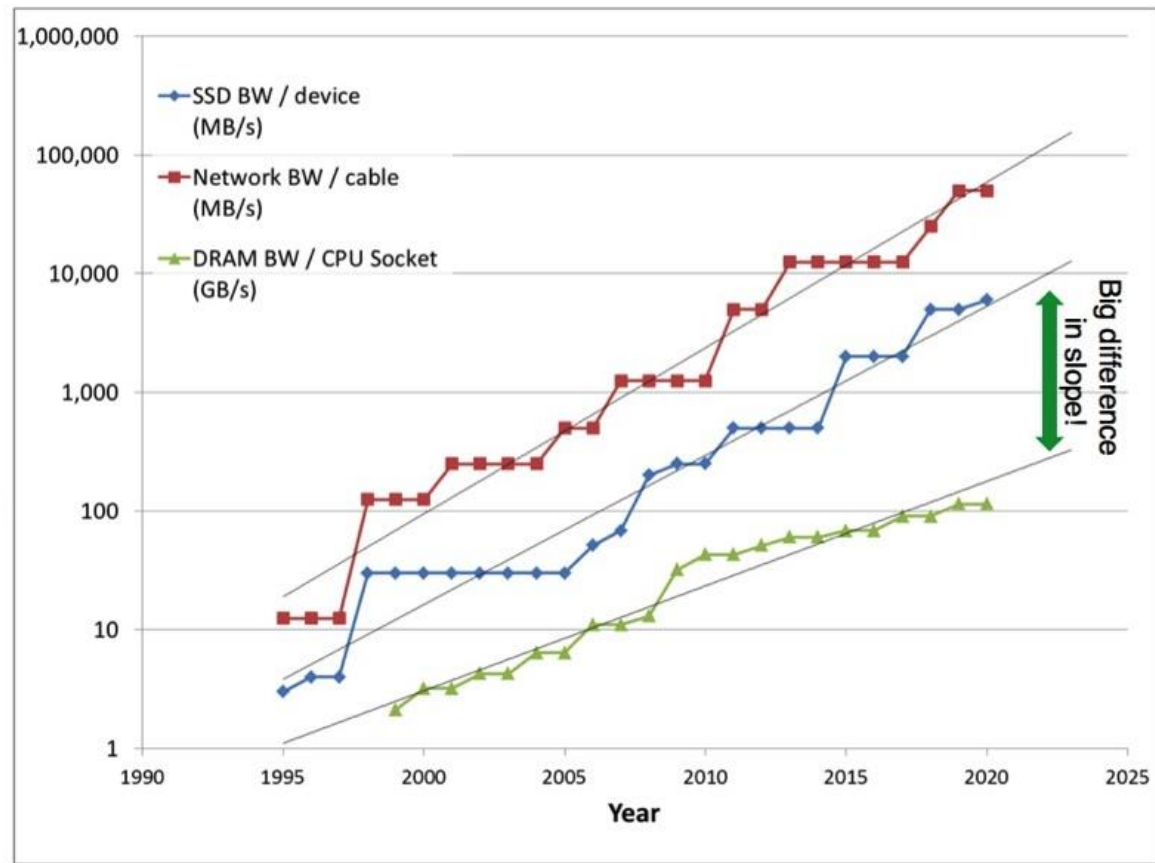
Key stroke ~100 ms

Network, IO, and Memory Bandwidth Trends

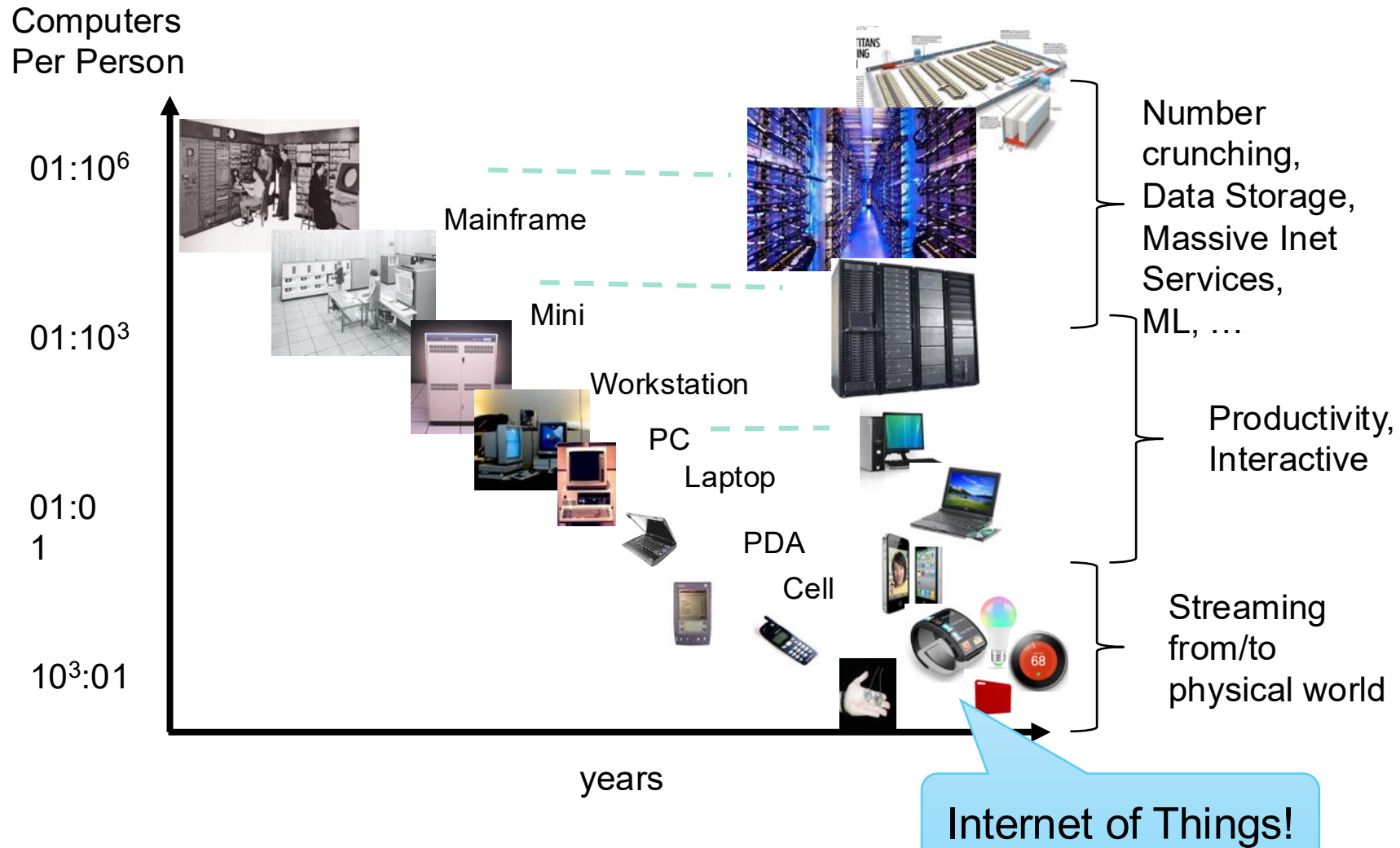
Network, Storage, & DRAM trends

Log scale

- Use DRAM Bandwidth as a proxy for CPU throughput
- Reasonable approximation for DMA and poor cache performance workloads (e.g. Storage)



People to Computer Ratio Trend



Summary

- OS provides VM abstraction to handle diverse HW
 - OS simplifies application development by providing standard services
- OS coordinates resources and protect users from each other
 - OS can provide fault containment, fault tolerance, and fault recovery
- ECE 350 combines ideas and concepts from many other areas of computer science and engineering
 - Languages, data structures, hardware, and algorithms

Questions?



Acknowledgment

- Slides by courtesy of Anderson, Culler, Stoica, Silberschatz, Joseph, Canny, and Kumar (Sam)